

SV LAB CODES

EXPERIMENT-1

```
module VariableDeclarationExample;

// Declare integer variable
int integerVariable;

// Declare 4-bit vector variable
logic [3:0] vectorVariable;

// Declare single-bit variable
bit singleBitVariable;

// Declare register variable
logic registerVariable;

// Declare 2D array
int arrayExample [0:3][0:7]

// Module always block for simulation purposes
initial begin

    // Assign values to variables
    integerVariable = 42;
    vectorVariable = 4'b1101;
    singleBitVariable = 1;
    registerVariable = 0;

    // Accessing array elements
    arrayExample[0][0] = 10;
    arrayExample[1][2] = 5;

    // Display variable values
    $display("Integer Variable: %0d", integerVariable);
    $display("Vector Variable: %b", vectorVariable);
    $display("Single Bit Variable: %b", singleBitVariable);
    $display("Register Variable: %b", registerVariable);
```

```

// Display array elements

$display("Array Example[0][0]: %0d", arrayExample[0][0]);

$display("Array Example[1][2]: %0d", arrayExample[1][2]);

end

endmodule

```

EXPERIMENT-2

```

module PackedUnpackedArraysExample;

// Packed array (continuous memory)
logic [3:0][7:0] packedArray;

// Unpacked array (separate elements)
logic [7:0] unpackedArray [0:3];

initial begin

// Assign values to packed array
packedArray[0] = 8'b11001100;
packedArray[1] = 8'b10101010;

// Assign values to unpacked array
unpackedArray[0] = 8'b11001100;
unpackedArray[1] = 8'b10101010;

// Display packed array
$display("Packed Array[0]: %b", packedArray[0]);
$display("Packed Array[1]: %b", packedArray[1]);

// Display unpacked array
$display("Unpacked Array[0]: %b", unpackedArray[0]);

```

```
$display("Unpacked Array[1]: %b", unpackedArray[1]);  
end  
endmodule
```

EXPERIMENT-3

```
module tb_4H5;  
string fruits[$] = {"grape","apple","mango","banana"};  
initial begin  
    // size()  
    $display("Number of fruits=%0d fruits=%p",fruits.size(),fruits);  
    // insert()  
    fruits.insert(1,"peach");  
    $display("Insert peach,size=%0d fruits=%p",fruits.size(),fruits);  
    // delete()  
    fruits.delete(3);  
    $display("After delete,size=%0d fruits=%p",fruits.size(),fruits);  
    // pop_front()  
    $display("Pop front %s,size=%0d fruits=%p",fruits.pop_front(),fruits.size(),fruits);  
    // push_front()  
    fruits.push_front("apricot");  
    $display("Push front,size=%0d fruits=%p", fruits.size(),fruits);  
    // pop_back()  
    $display("Pop back %s,size=%0d fruits=%p", fruits.pop_back(),fruits.size(),fruits);  
    // push_back()  
    fruits.push_back("plum");
```

```

    $display("Push back,size=%0d fruits=%p",fruits.size(),fruits);
end
endmodule

```

EXPERIMENT-4

```

module tb_4H5();
//Create two dynamic arrays of type int int array [];
int id[] ;
int array[];
initial begin
// Allocate 5 memory locations to "array" and initialize with values
array = new [5];
array = '{1,2,3,4,5}';
id = array;           //Point "id" to "array"
$display ("id = %p", id);    //Display contents of "id"
id = new [id.size() + 1] (id);
id [id.size() - 1] = 6;      //Assign value 6 to the newly added location [index 5]
$display ("New id = %p", id); //Display contents of new "id"
$display ("array size = %0d,id size = %0d",array.size(), id.size());
//Display size of both arrays
end
endmodule

```

EXPERIMENT-5

```
module Example;

reg [3:0] data;

initial begin

    // Initialize data

    data = 4'b0000;

    // Perform some operations on data

    data = data + 4'b0011;

    // Immediate assertion to check the value of data

    assert (data == 4'b0011)

        $display ("Data equal to 0011");

        else $fatal("Assertion failed: data is not equal to 4'b0011");

    // Continue with the simulation

    // Another immediate assertion

    assert (data == 4'b0100)

        $display ("Data equal to 0001");

        else $fatal("Assertion failed: data is not equal to 4'b0100");

    $display("Simulation completed successfully!");

    $finish;

end

endmodule
```

EXPERIMENT-6

```
module tb;

    bit a, b;
    bit clk;

    // Clock initialize
    initial clk = 0;
    // Clock generation (period = 10)
    always #5 clk = ~clk;
    // Stimulus
    initial begin
        repeat (10) begin
            @(posedge clk);
            a <= $random;
            b <= $random;
            $display("[%0t] a=%0b b=%0b", $time, a, b);
        end
        #10 $finish;
    end

    // concurrent assertion
    assert property (@(posedge clk) (a || b))
        else $error("Both are 0 at time %0t", $time);
endmodule
```

EXPERIMENT-7

TASK

```
module sv_task_4H5;

    int x;

    task mul(input int a, b, output int c);
        c = a * b;
    endtask

    initial begin
        mul(10, 5, x);
        $display("Value of x = %0d", x);
    end
endmodule
```

FUNCTION

```
module tb;

    int res;
    bit [7:0] s;

    // Function: sum
    function int sum(input int x, y);
        return x + y;
    endfunction

    // Function: mul
    function int mul(input int x, y);
        return x * y + 1;
    endfunction

    initial begin
        s = sum(5, 9);
```

```

$display("s = %0d", s);

$display("sum(5,9) = %0d", sum(5, 9));

res = mul(3, 1);

$display("mul(3,1) = %0d", res);

$display("res = %0d", res);

end

endmodule

```

EXPERIMENT-8

Pass By Value

```

module argument_passing_value;

  int x, y, z;

  function int sum(int x, y);

    x = x + y;

    return x + y;

  endfunction

  initial begin

    x = 20;

    y = 30;

    z = sum(x, y);

    $display("---- PASS BY VALUE ----");

    $display("x = %0d", x); // unchanged

    $display("y = %0d", y); // unchanged

    $display("z = %0d", z);

  end

endmodule

```


PASSING ARGUMENTS BY REFERENCE

```
module argument_pass_reference;

    int x,y,z;

    //function to add two integer numbers.

    function int sum( int x,y);

        x = x+y;

        return x+y;

    endfunction

    initial begin

        x = 20;

        y = 30;

        z = sum(x,y);

        $display("\tValue of x = %0d",x);

        $display("\tValue of y = %0d",y);

        $display("\tValue of z = %0d",z);

    end

endmodule
```

EXPERIMENT-9

```
class Packet;

    int count;

endclass

module tb;

    Packet pkt; // handle (points to object)

    initial begin

        if (pkt == null)

            $display("Packet handle 'pkt' is null");

    end

endmodule
```

```

// Create object
pkt = new();
if (pkt == null)
    $display("What's wrong, pkt is still null?");
else
    $display("Packet handle 'pkt' is now pointing to an object");
// Access member
$display("count = %0d", pkt.count);
end
endmodule

```

EXPERIMENT-10

```

class parent_class;
    bit [31:0] addr;
    function void display();
        $display("Addr = %0d", addr);
    endfunction
endclass

class child_class extends parent_class;
    bit [31:0] data;
    function void display();
        super.display(); // call parent function
        $display("Data = %0d", data);
    endfunction
endclass

```

```

module inheritance_4H5;
    initial begin
        child_class c = new();
        c.addr = 10;
        c.data = 20;
        c.display();
    end
endmodule

```

EXPERIMENT-11

```

class parent_class;
    bit [31:0] addr;
    function void display();
        $display("Addr = %0d", addr);
    endfunction
endclass

class child_class extends parent_class;
    bit [31:0] data;
    function void display();
        super.display();
        $display("Data = %0d", data);
    endfunction
endclass

module inheritance_4H5;
    initial begin

```

```

parent_class p;
child_class c = new();
child_class c1;
c.addr = 10;
c.data = 20;
p = c; // upcasting
// casting check
if ($cast(c1, p)) begin
    c1.display();
end else begin
    $display("Casting failed");
end
end
endmodule

```

EXPERIMENT-12

```

class A;
    int i;
endclass

class B;
    A a;
endclass

module main_4H5;
    initial begin
        B b1;
    end
endmodule

```

```

B b2;

b1 = new();

b1.a = new();

b1.a.i = 123;

b2 = new b1; //shallow copy

$display(b1.a.i);

$display(b2.a.i);

b1.a.i = 321;

$display(b1.a.i);

$display(b2.a.i);

end

endmodule

```

EXPERIMENT-13

```

interface v2f (input logic clk, rst);

    logic [3:0] a, b;

    logic [4:0] sum;

    // clocking block (TB side)

    clocking cb_tb @(posedge clk);

    default input #1step output #1;

    output a, b;

    input sum;

endclocking

// modports

modport TB (clocking cb_tb, input clk, rst);

modport DUT (input clk, rst, a, b, output sum);

endinterface

```

```
module adder (v2f.DUT vif);  
    always @(posedge vif.clk or posedge vif.rst) begin  
        if (vif.rst)  
            vif.sum <= '0;  
        else  
            vif.sum <= vif.a + vif.b;  
        end  
    endmodule
```

```
module tb;  
    logic clk, rst;  
    // interface instance  
    v2f vif(clk, rst);  
    // connect DUT using modport  
    adder dut(vif);  
    // clock generation  
    always #5 clk = ~clk;  
    initial begin  
        clk = 0;  
        rst = 1;  
        vif.a = 0;  
        vif.b = 0;  
        #10 rst = 0;  
        @(posedge clk);  
        // direct driving (no clocking block)  
        vif.a = 4'd5;  
        vif.b = 4'd10;  
        @(posedge clk);
```

```

$display("SUM = %0d", vif.sum);
#10 $finish;
end
endmodule

```

EXPERIMENT-14

```

module FSM (clk,reset_n,x,z);
    input clk;
    input reset_n;
    input x;
    output reg z;
    // State encoding
    parameter A = 2'b00;
    parameter B = 2'b01;
    parameter C = 2'b10;
    parameter D = 2'b11;
    reg [1:0] state, next_state;
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n)
            state <= A;
        else
            state <= next_state;
    end
    always @(*) begin
        case (state)
            A: begin
                next_state = x ? B : A;

```

```

        z = 0;
    end

    B: begin
        next_state = x ? B : C;
        z = 0;
    end

    C: begin
        next_state = x ? D : A;
        z = 0;
    end

    D: begin
        next_state = x ? B : C;
        z = x ? 1 : 0; // 1011 detect
    end

    default: begin
        next_state = A;
        z = 0;
    end

endcase

end

endmodule

module tb;

    logic clk, reset_n, x;

    logic z;

    // Instantiate FSM
    FSM dut (clk, reset_n, x, z);

    // Clock generation
    always #5 clk = ~clk;

```



```

initial begin
    clk = 0;
    reset_n = 0;
    x = 0;
    // Reset
    #10 reset_n = 1;
    @(posedge clk) x = 1;
    @(posedge clk) x = 0;
    @(posedge clk) x = 1;
    @(posedge clk) x = 1;
    @(posedge clk) x = 0;
    @(posedge clk) x = 1;
    @(posedge clk) x = 1;
    #20 $finish;
end

// Monitor
initial begin
    $monitor("Time=%0t x=%0b z=%0b", $time, x, z);
end

endmodule

```

EXPERIMENT-15

```

module adder(
    input  logic [3:0] a, b,
    output logic [4:0] sum
);

```

```

        assign sum = a + b;
    endmodule

interface adder_if;
    logic [3:0] a, b;
    logic [4:0] sum;
endinterface

class transaction;
    rand bit [3:0] a, b;
    bit [4:0] sum;
    function void display(string name);
        $display("[%s] a=%0d b=%0d sum=%0d", name, a, b, sum);
    endfunction
endclass

class generator;
    mailbox gen2drv;
    function new(mailbox gen2drv);
        this.gen2drv = gen2drv;
    endfunction
    task run();
        transaction tx;
        repeat(5) begin
            tx = new();
            assert(tx.randomize());
            gen2drv.put(tx);
        end
    endtask

```

```
endclass
```

```
class driver;
```

```
    virtual adder_if vif;
```

```
    mailbox gen2drv;
```

```
function new(mailbox gen2drv, virtual adder_if vif);
```

```
    this.gen2drv = gen2drv;
```

```
    this.vif = vif;
```

```
endfunction
```

```
task run();
```

```
    transaction tx;
```

```
    forever begin
```

```
        gen2drv.get(tx);
```

```
        vif.a = tx.a;
```

```
        vif.b = tx.b;
```

```
        #1;
```

```
    end
```

```
endtask
```

```
endclass
```

```
class monitor;
```

```
    virtual adder_if vif;
```

```
    mailbox mon2scb;
```

```
function new(mailbox mon2scb, virtual adder_if vif);
```

```
    this.mon2scb = mon2scb;
```

```
    this.vif = vif;
```

```
endfunction
```

```
task run();
```

```
    transaction tx;
```

```
    forever begin
        tx = new();
        #1;
        tx.a = vif.a;
        tx.b = vif.b;
        tx.sum = vif.sum;
    mon2scb.put(tx);
    end
endtask
endclass
```

```
class scoreboard;
    mailbox mon2scb;
    function new(mailbox mon2scb);
        this.mon2scb = mon2scb;
    endfunction
    task run();
        transaction tx;
        forever begin
            mon2scb.get(tx);
            if (tx.sum == tx.a + tx.b)
                $display("PASS: a=%0d b=%0d sum=%0d", tx.a, tx.b, tx.sum);
            else
                $display("FAIL: a=%0d b=%0d sum=%0d", tx.a, tx.b, tx.sum);
            end
        endtask
    endclass
```

```
class environment;
```

```

generator gen;

driver drv;

monitor mon;

scoreboard scb;

mailbox gen2drv;

mailbox mon2scb;

virtual adder_if vif;

function new(virtual adder_if vif);

    this.vif = vif;

    gen2drv = new();

    mon2scb = new();

    gen = new(gen2drv);

    drv = new(gen2drv, vif);

    mon = new(mon2scb, vif);

    scb = new(mon2scb);

endfunction

task run();

    fork

        gen.run();

        drv.run();

        mon.run();

        scb.run();

    join_any

endtask

endclass

module tb;

    adder_if vif();

    adder dut(

```

```
.a(vif.a),  
.b(vif.b),  
.sum(vif.sum)  
);  
environment env;  
initial begin  
env = new(vif);  
    env.run();  
    #20 $finish;  
end  
endmodule
```